

Fleet Collapse

A Python runtime layer that makes one process do the work of thirty — and lets you debug what's left.

Project plan v6 · 12 September 2026 · Written for: the project owner and the engineers who will build it

1. The Thesis

A typical Python deployment runs dozens of near-identical worker processes because the GIL left no alternative. Each duplicates the interpreter, the imports and the application. The fleet is sized for that duplication, and everything downstream — node count, the cluster to schedule them, the platform team to run the cluster — follows from it.

Collapse the duplication and the rest of the stack collapses with it. That is the product. Kubernetes removal is a consequence of needing three nodes instead of ten, not a goal in its own right.

In one line: make one Python process do the work of thirty, automatically, and make the failures in it debuggable across machines.

WHY THIS FRAMING AND NOT THE PREVIOUS ONE

An earlier draft led with replacing Kubernetes. Costing the claim honestly showed that of a ~72 % infrastructure reduction, roughly 90 % comes from node consolidation and about 10 % from removing the control plane. A team could capture nearly all of the saving by fixing their concurrency model and leaving Kubernetes exactly where it is. Leading with the cluster would have been selling the smaller half.

WHO THIS IS FOR

The boundary is not company size — it is whether the organisation has a platform engineering team that builds its own tooling. A 5,000-person company with no Python platform group is an excellent fit. A 500-person company that has written its own scheduler is not, and never will be: at that point they build rather than adopt, which is exactly how Cinder and Peloton came to exist. Zero configuration is worthless to an organisation whose infrastructure team needs control knobs, custom schedulers and chargeback models.

BEACHHEAD: NOTEBOOK AND ML PLATFORMS

JupyterHub deployments, ML platform teams and data-science orgs are the sharpest early target. A single JupyterHub can run hundreds of kernels, each a full interpreter holding its own copy of the scientific stack, and most of them idle most of the time. The waste is obvious, the before-and-after number is unambiguous, and no incumbent addresses it.

A different lever applies here, and it is smaller. Notebook kernels execute untrusted user code with arbitrary imports, so they need process-level isolation — which means the free-threaded thread tier that carries the \$5 collapse *cannot* be shared across users. Two measured findings close the other doors: subinterpreters share no module memory, and numpy aborts the process under them. Kernels stay processes.

What does apply is the warm-parent fork pool of \$7, used as **kernel prewarming**: a parent imports the common stack once, calls `gc.freeze()` to keep the collector from touching those objects, and forks a kernel

per user. Copy-on-write then shares the library pages until something writes to them — Instagram's production technique, applied to kernels. Two benefits follow:

- **Shared library pages across kernels** rather than one private copy each. The saving scales with how heavy the common stack is and how many kernels are idle.
- **Near-instant kernel start** — 0.7 ms to fork against multi-second cold imports of the data stack. A user-visible improvement independent of any memory saving.

Do not carry the headline number here. This is not the 60–80 % node collapse; it is a different and smaller saving from a different mechanism. Its size is unmeasured: copy-on-write sharing cannot be quantified reliably on macOS — `vm_stat` reported 4.25 MB for a process independently measured at 26 MB, because memory compression confounds it. Measure on Linux with `/proc/smaps_rollup` PSS before quoting any figure.

2. What We Build and What We Wrap

Three of the problems in this space are real and already solved by good projects. Competing with them wins nothing and costs everything.

COMPONENT	DECISION	RATIONALE
Task queue	Wrap	SAQ and streaq are async-native, fast and maintained. We will not beat them, and beating them would win nothing.
WebSocket server	Wrap	Granian's Rust core already solves frame handling. Same reasoning.
Cluster scheduling	Out	Bin-packing, quotas and multi-tenancy exist for good reasons. Beyond our boundary, use Nomad.
Automatic concurrency	Build	Free-threading, subinterpreters and supervision exist as primitives but nobody has packaged them. This is the entire cost saving.
Distributed debugging	Build	Genuinely unsolved everywhere, including inside Meta and Uber. The only real moat.
Zero-config integration	Build	Assembling FastAPI + Celery + Redis + Flower + a scheduler + a WS layer + manifests is a real tax nobody has removed.

Consequence for the roadmap. Wrapping rather than rebuilding removes roughly six weeks of work that would have been spent losing to incumbents, and lets the concurrency engine and the debugger start in week one.

3. Zero Configuration Is the Delivery Mechanism

Governing principle. Everything here must be automatic for the developer who installs the package. If a feature needs a config file, a broker to provision, a worker CLI, a YAML document or a deployment descriptor, it is not finished. Escape hatches exist for the 5 % who need them; the other 95 % write decorators and nothing else.

The concurrency engine only pays off if it is invisible. A developer who must reason about subinterpreters, thread pools and free-threaded builds will not get the collapse — so the runtime decides, and the developer writes:

```
from orchestra import task, actor, socket, serve

@task                                # background work – queued, retried, durable
async def charge(order_id: int) -> Receipt: ...

@actor(isolated=True)                # own heap, own GC, crash-contained
class PricingEngine: ...

@socket("/live")                      # realtime endpoint
async def live(ws): ...

serve()                               # HTTP + WebSocket + workers + scheduler, one process tree
```

CONCERN	AUTOMATIC BEHAVIOUR
Interpreter build	Free-threaded and GIL builds detected at import. Free-threaded → one process, threads sized to cores. GIL build → pre-forked processes. Identical semantics either way.
Concurrency tier	Each unit placed on a subinterpreter, thread or coroutine by its declared characteristics — see §5. Never a developer decision.
Queue backend	Wrapped SAQ or streaq. In-process SQLite by default — nothing to install. Detects <code>DATABASE_URL</code> or <code>REDIS_URL</code> and upgrades transparently.
Realtime transport	Granian mounted automatically by <code>serve()</code> . Cross-node broadcast over whatever backend is already present; no pub/sub to configure.
Pool sizing	Derived from core count and measured task profile; memory ceiling and recycling built in. No worker CLI as a separate deployment.
Local vs distributed	No hosts configured → multi-process locally. Hosts appear → same code distributes, identical semantics.
Observability	OpenTelemetry auto-instrumented when a collector exists; structured logs with causal task IDs otherwise.
Every automatic decision	Logged at startup and overridable. <code>orchestra doctor</code> prints what was chosen and why.

4. Targets and the Cost Model

Illustrative mid-size deployment: 20 Python services, managed Kubernetes, 10 × m5.xlarge workers.

LINE	BEFORE	AFTER	SAVING
Worker nodes	\$1,400/mo	~\$420/mo	\$980 — node consolidation
Managed control plane	\$73/mo	\$0	\$73
Redis for Celery	\$50/mo	\$0	\$50
Node agent overhead	~2% of nodes	0	~\$30
Infrastructure	~\$1,520/mo	~\$420/mo	~72 %
Platform engineer time	~0.3 FTE	~0.1 FTE	~\$30k/yr

Read the first row carefully. \$980 of the \$1,100 is node consolidation; the control plane is about \$100. This is why §1 leads with fleet collapse. It is also why Phase 2 exists: the consolidation claim must be evidenced on a real application before anything else is built on top of it.

ENGINEERING TARGETS

METRIC	TARGET	BASIS
Node count for a fixed workload	–60 to –80 %	The headline claim; proven in Phase 2
Worker idle RSS	≤ 30 MB	Measured: interpreter + <code>asyncio.start_server</code> = 21.2 MB
Isolation domains / node	1 per core	Measured 2,336 KB each; 64 cores ≈ 150 MB
Preemptible workers / node	10k+	Measured 35 KB per thread at 256 KB stack
I/O concurrency / node	100k+	Measured 0.92 KB per asyncio task
Task dispatch, local	< 1 ms	Measured: <code>fork()</code> from warm parent = 0.7 ms
Task dispatch, remote	< 5 ms	Parity with wrapped SAQ
RAM under large payloads	Flat	Streaming with backpressure

Not a target. "A million times faster" is unachievable and chasing it produces bad decisions. Application code runs at exactly the same speed — every saving here is overhead and duplication. The one legitimate order-of-magnitude claim: replacing a Celery + Redis round-trip (1–5 ms) with in-process dispatch (microseconds) is a real 100–1000× latency reduction for small tasks.

5. Concurrency Model — the Engine

The design target is the BEAM, the runtime behind Erlang and Elixir, which achieves its efficiency through four properties: a shared runtime with no per-process duplication, extremely cheap concurrency units, preemptive scheduling that lets no unit monopolise a core, and per-process heaps with isolated collection and supervision. Three of the four are reachable in Python today. The fourth is a density limit, not a missing capability.

MEASURED PRIMITIVES

Python 3.13.1, macOS arm64, direct measurement:

UNIT	MEMORY	SPAWN	OWN HEAP	OWN GC	PREEMPTIVE
BEAM process (<i>reference</i>)	~2 KB	~µs	Yes	Yes	Yes
Subinterpreter	2,336 KB	8.37 ms	Yes	Yes	Yes — on its own OS thread
Thread (256 KB stack)	35 KB	0.032 ms	No	No	Yes — kernel-scheduled
asyncio task	0.92 KB	0.0026 ms	No	No	No — cooperative

An asyncio task is *cheaper* than a BEAM process. A subinterpreter has every semantic property of one. No single primitive has both — which dictates the tiered model below.

PREEMPTION IS AVAILABLE; THE MISTAKE IS OPTING OUT OF IT

Threads and subinterpreters are scheduled by the kernel. They are genuinely preemptive: no amount of tight looping lets one starve another. Only bare coroutines are cooperative, and that is a choice we make rather than a limit Python imposes.

Architectural rule. Nothing that might compute runs as a bare coroutine. The event loop is reserved for I/O; anything CPU-bound is routed to a thread or subinterpreter automatically. This single rule buys back the preemption guarantee async Python is usually assumed to lack.

Literal reduction counting is buildable where the guarantee is needed even for cooperative units.

`sys.monitoring` (PEP 669) exposes `BRANCH` and `JUMP` events, made cheap by PEP 659 quickening — that is reduction counting. Paired with `greenlet`, which can switch stacks from any point rather than only at `await`, a hook forces a switch when the budget is exhausted: BEAM's mechanism, in Python. It carries real overhead, so it ships as opt-in strict mode for untrusted or unpredictable code, never as the default.

ISOLATION: SUBINTERPRETERS ARE THE ACTOR PRIMITIVE

PEP 734 shipped in Python 3.14 as `concurrent.interpreters`, built on PEP 684's per-interpreter GIL. Each subinterpreter provides its own memory arenas — a genuinely separate heap; its own GIL and garbage collector, so collecting one never pauses the others; and message passing only, via pickle-copied queues, the buffer protocol, or immortal immutables. Shared mutable state is not discouraged by convention — it is impossible by construction.

That last property is the same enforcement BEAM has, arrived at by the same design: BEAM also copies messages between processes, refcounting only large binaries, precisely mirroring subinterpreter queues with buffer-protocol sharing for large payloads.

WHICH PRIMITIVE ACTUALLY COLLAPSES THE FLEET

Measured correction. Subinterpreters do *not* share module memory — each re-imports into its own namespace. Twenty subinterpreters importing only `json` cost 3.60 MB each against 2.34 MB empty: +1.26 MB apiece for one small stdlib module. A library-heavy workload pays nearly full process price per subinterpreter.

Worse, **numpy 2.3.0 aborts the entire process when imported into a subinterpreter** on Python 3.13 — `Fatal Python error: Aborted`, SIGABRT, exit 134. Not a catchable exception: the host process dies.

This settles which tier does the work. **Free-threading is the collapse mechanism.** Threads share one heap and one set of module objects, so twenty threads using numpy hold one copy of it — one process at 26 MB rather than twenty at 26 MB. That is the 60–80 % claim, and it rests on threads, not subinterpreters.

Subinterpreters are an isolation feature, not a memory saving, and today they are unavailable to the numpy ecosystem entirely. They earn their place for domains running application code and I/O where crash containment and per-heap collection matter and the dependency tree permits them. They are conditional, not foundational.

THE TIERED MODEL

A subinterpreter costs 2,336 KB and 8.37 ms even before imports, so it cannot be an actor — it must be an **isolation domain**:

Subinterpreter	one per core, tenant or service	tens to low hundreds
Thread	preemptible worker inside it	thousands
asyncio task	I/O concurrency	100k+ (0.92 KB each)

Sixty-four empty subinterpreters cost roughly 150 MB, but only where the dependency tree supports them — the runtime probes this at startup and silently falls back to threads where it does not. The thread tier is always available and always carries the collapse; the subinterpreter tier adds isolation when it can. The tier is chosen by the runtime, never by the developer.

SUPERVISION TREES

OTP's supervision model is implementable over whichever tier is in use: `one_for_one`, `one_for_all` and `rest_for_one` restart strategies, restart intensity limits, and let-it-crash as the primary error strategy instead of defensive exception handling. Containment is strongest over subinterpreters, where a crashed domain's heap is reclaimed outright; over threads it is weaker — a hard crash can still take the process — so on the thread tier supervision restarts a worker within a process that is itself supervised by systemd. The API is identical; only the blast radius differs.

HONEST SCORECARD AGAINST THE BEAM

PROPERTY	STATUS	NOTE
Shared runtime, no duplication	Reached	Free-threaded build: one heap across threads
Preemptive scheduling	Reached	Kernel scheduling of threads and subinterpreters; strict mode available
Per-heap GC and fault isolation	Reached	Subinterpreters, at domain rather than actor granularity
Supervision and let-it-crash	Reached	Implementable as a library; OTP semantics
Density with isolation	Not reached	Thousands, not millions. 2,336 KB vs 2 KB. Unavoidable.
Spawn cost	Not reached	8.37 ms forces pooling; never spawn per message
Isolation for numeric workloads	Blocked	numpy aborts the process under subinterpreters. Threads-only tiering for the scientific stack today.
Hot code reload	Not reached	Out of scope

The shortfall is a number rather than a missing capability — the difference between approximating the model and adopting it.

6. The Debugger — the Moat

When distributed Python fails, the developer gets decoupled worker logs, serialization errors and unexplained OOM kills. This is unsolved everywhere, at every scale, and it is the only component here that no incumbent supplies.

- **Traceback reconstruction.** A remote exception is serialized with its frames and frame-local variables and re-raised locally so it reads like a local stack trace, with correct chaining through `__cause__` and `__context__`.
- **Causal task IDs** threaded through every log line and span, so one failure is traceable across hosts and tiers without correlating timestamps by hand.
- **Kill diagnosis.** When the kernel OOM-kills a worker or a domain dies hard, report what it was executing, its memory trajectory and its last messages — the case where existing tools give nothing at all.
- **Live introspection.** Every in-flight unit, its tier, its state and its stack, always available. The equivalent of Erlang's `:observer`, which Python has never had.

Scope discipline. Traceback reconstruction and introspection ship first. Live remote stepping is a separate, later decision — it is where this feature would otherwise expand without limit.

7. Cross-Cutting Architecture

EXECUTION MODEL

The tiers of §5 rest on a warm pool sized to the detected interpreter build: threads on free-threaded builds, pre-forked processes on GIL builds — a warm parent imports the application once, then forks. Copy-on-write makes this cheap.

```
fresh interpreter (posix_spawn path): 15.0 ms/task
+ a few stdlib imports:                27.2 ms/task
fork() from warm parent:                0.7 ms/task    (21x cheaper)
```

Spawn-per-task is rejected: it pays full interpreter startup *and* re-imports everything on every call. At 1000 tasks/sec that is 15 CPU-seconds per second before any work happens.

Free-threaded builds are a first-class target, officially supported since 3.14 under PEP 779. Single-threaded cost is 1–8 % by platform; multi-core gains reach ~3.5× on four cores. Ecosystem support remains mixed, so the library runs correctly on both builds and detects which it is on.

SERIALIZATION

`cloudpickle` for functions and closures; **pickle protocol 5 with out-of-band buffers** for large payloads, giving true zero-copy transfer.

marshal is rejected — verified by direct test. It cannot serialize `datetime`, `Decimal` or any user-defined class instance. A marshalled code object loses module globals (`NameError`) and closures (`TypeError: arg 5 (closure) must be tuple`). Its format is version-specific and undocumented for transport, and it is explicitly unsafe against malicious data. It is strictly less capable than pickle — and pickle 5's out-of-band buffers make it faster for large data as well.

DISTRIBUTION AND AUTHENTICATION

Length-prefixed binary frames over raw TCP on a WireGuard mesh. Not SSH-multiplexed — SSH's per-channel flow-control window causes head-of-line blocking. Not WebSocket, which exists to traverse HTTP proxies and reach browsers and is pure overhead between our own processes. Workers self-register with heartbeats; dead nodes are reaped and their in-flight work redelivered.

Authentication is non-negotiable and ships with the first line of remote execution. A worker executing payloads from a socket is a remote-code-execution service. Required: mTLS or pre-shared key with per-message HMAC; listener bound to the WireGuard interface, never `0.0.0.0`; no execution path reachable before authentication succeeds. A benefit of the revised phase order is that no remote-execution surface exists until Phase 5, so this cannot be deferred past the feature that needs it.

DEPLOYMENT

No container requirement. Bare processes under systemd by default — `Restart=always`, `MemoryMax=`, `CPUQuota=`, `ProtectSystem=strict`, the same cgroups Docker uses with no daemon in front. Running inside existing containers or an existing Kubernetes pod is fully supported; requiring them is not. A library that mandates a runtime is a platform, and platforms are what we are collapsing.

The honest cost is native dependencies: `uv` and lockfiles solve pure-Python and manylinux wheels, not glibc, libpq, CUDA or ffmpeg. Answer: pin the machine image rather than the container image, verify interpreter and lockfile hash at worker registration so skew is a startup error rather than a corrupted result, and keep containers as the documented path for heavy native and GPU workloads.

MEMORY DISCIPLINE

Generator-based streaming with backpressure so RAM stays flat regardless of payload size; `mmap` and `multiprocessing.shared_memory` for read-only datasets shared across workers. `gc.freeze()` after import to preserve copy-on-write sharing; jemalloc and `MALLOC_ARENA_MAX` tuned by default.

8. Competitive Position

AREA	OUR STANCE	REASONING
SAQ, streaq, Taskiq, Procrastinate	Wrap	Mature and async-native. We add zero provisioning and integration, not a fourth queue.
Granian, uWebSockets	Wrap	Rust cores already solve frame handling.
Ray, Dask, Runhouse	Differentiate	They require a cluster runtime alongside the app. We are a library inside it.
Kubernetes, Nomad	Sidestep	We reduce the node count that made a scheduler necessary. We do not schedule.
Celery	Displace	Via the wrapped modern queues, not by rewriting it.
Automatic concurrency tiering	Uncontested	Every primitive is public; nobody has packaged them together.
Distributed Python debugging	Uncontested	Unsolved at every scale, Meta and Uber included.
Notebook kernel consolidation	Uncontested	JupyterHub culls idle kernels but does not share their memory. Prewarming exists in scattered form, never packaged.

Where we lose, and it is fine. Organisations with platform teams that write their own schedulers will build rather than adopt — that is how Cinder and Peloton exist. Uber completed a migration to Kubernetes in 2024 and runs Go and Java in its hot paths; Instagram runs a CPython fork with its own JIT on Meta's internal platform. Neither is a customer. The market is the far larger middle: teams running Python who adopted a cluster because it was the default, not because they hit a constraint that required one.

9. Adoption Path

1. **Inside the existing deployment.** Import the library into a service already running under Kubernetes. Concurrency tiering collapses its worker count; the debugger works immediately. Nothing about the deployment changes.
2. **Absorb the sidecars.** Background workers that were separate Deployments collapse into the main process. Pod count and the manifests behind them start falling.
3. **Shrink the fleet.** Fewer, smaller nodes for the same workload. This is where the money in §4 is realised.
4. **Leave the cluster.** Optional. VMs under systemd, still running the organisation's image. Control plane and node agents disappear.
5. **Drop the image.** Optional, per service, only where native dependencies are light.

Each step is independently valuable and independently reversible. **Large organisations will realistically stop at step 2 or 3** — and that is still a real product for them. Steps 4 and 5 are for teams without a platform group to defend.

10. Build Plan

Reordered from v3 so the cost claim is evidenced at week 7 rather than week 19.

Phase 0 — Validate the wrap-or-build split 1 WEEK

- Benchmark SAQ, streaq and Granian against \$4 to confirm wrapping beats rebuilding
- **C-extension compatibility spike.** Partially run already: numpy 2.3.0 aborts the process under subinterpreters on 3.13, and subinterpreters share no module memory. Complete the matrix across the real dependency tree, on both 3.13 and 3.14, and verify free-threaded wheels for every library on the hot path.

Exit: a compatibility matrix naming which dependencies permit the subinterpreter tier, and a decision on each wrapped component — before anything is designed around either.

Phase 1 — Concurrency engine THE PRODUCT 4 WEEKS

- Warm pool auto-sized to the detected build; memory ceiling; recycle-after-N-tasks
- **Free-threaded thread tier first** — this is what delivers the collapse and must work before anything else
- Subinterpreter isolation domains where the dependency tree permits, pooled — probed at startup, silent fallback to threads
- Automatic tier routing across subinterpreter, thread and coroutine
- Supervision trees: `one_for_one`, `one_for_all`, `rest_for_one`, restart intensity limits
- Both interpreter builds validated in CI

Exit: < 1 ms local dispatch, ≤ 30 MB idle worker, and a domain killed mid-task restarts without disturbing siblings.

Phase 2 — Prove the collapse

THE CLAIM

3 WEEKS

Case A — web service, thread tier. The headline claim.

- Port a representative real application onto the engine
- Measure node count, total RSS and throughput before and after, under identical load

Case B — JupyterHub, copy-on-write tier. The beachhead, exercising a different lever.

- Warm-parent kernel prewarming with `gc.freeze()` ; fork per user session
- Measure PSS per kernel on Linux via `/proc/smaps_rollup` , plus kernel start latency
- Compare against stock JupyterHub kernel spawning at the same session count

Both cases publish methodology alongside numbers. macOS is not an acceptable measurement platform for either — memory compression makes its counters unreliable for shared-page accounting.

Exit: a documented 60–80 % node reduction on Case A. **If Case A fails, the project's premise is wrong and the remaining phases should not be built.** Case B is a market signal, not a gate: a weak result narrows the beachhead without invalidating the thesis.

Phase 3 — Distributed debugger

THE MOAT

4 WEEKS

- Remote traceback reconstruction with frame-local variables and correct exception chaining
- Causal task IDs threaded through logs and spans
- OOM and hard-kill diagnosis
- Live introspection of every in-flight unit

Exit: a failure three hosts away reads like a local stack trace.

Phase 4 — Integration layer

3 WEEKS

- `@task` over wrapped SAQ or streaq; backend auto-detected from the environment
- `@socket` over Granian, auto-mounted by `serve()` ; cross-node broadcast over the existing backend
- `serve()` as the single endpoint: HTTP, WebSocket, workers and scheduler in one process tree

Exit: no configuration file anywhere in the developer's project.

Phase 5 — Distribution

3 WEEKS

- Authenticated length-prefixed TCP over WireGuard; mTLS or pre-shared-key HMAC; audit log
- Adversarial tests: unauthenticated connect, replayed frame, oversized frame, truncated frame
- Worker self-registration and heartbeat; dead-node reaping and redelivery
- Startup verification of interpreter and lockfile hash

Exit: an unauthenticated peer cannot cause code to execute, proven by test rather than inspection.

Phase 6 — Deployment and simulation 2 WEEKS

- `orchestra deploy` : code sync, lockfile resolution, systemd unit generation, rolling restart
- `orchestra doctor` : print every automatic decision and its reason
- Simulated multi-node cluster with injectable latency, drops and failures for local tests
- OpenTelemetry auto-instrumentation

Total: ~20 weeks. Value lands at week 7, when Phase 2 either proves the cost claim or kills the project.

11. Success Criteria

1. **A real application runs on 60–80 % fewer nodes** for the same workload, with published methodology — achieved on the thread tier alone, with no subinterpreter dependency.
2. A developer reaches a working application with zero configuration files — validated by observing engineers new to the library.
3. Worker idle RSS ≤ 30 MB; dispatch < 1 ms local, < 5 ms remote.
4. A CPU-bound unit cannot stall unrelated work on the same node — proven with a tight infinite loop alongside live traffic.
5. A crashed isolation domain is restarted by its supervisor without affecting siblings, and its heap is fully reclaimed.
6. A remote exception produces a locally readable traceback including frame variables.
7. An OOM-killed worker yields a diagnosis, not silence.
8. RAM stays flat while streaming a payload larger than worker memory.
9. An unauthenticated peer cannot execute code — proven by adversarial test.
10. The package installs and runs with no container runtime present.

12. Risks

RISK	SEVERITY	RESPONSE
The collapse does not materialise on real code	Critical	Phase 2 is a kill gate at week 7, before the debugger and integration work are built on the premise
Dependencies not subinterpreter-safe — confirmed for numpy	High	Measured, not hypothetical: numpy aborts the process. Threads-only tiering is therefore the default path, losing per-heap GC while keeping preemption and <i>all</i> of the collapse. Subinterpreters are an opt-in bonus, never a dependency.
Free-threading ecosystem gaps	High	Both builds supported and tested; GIL build with pre-forked processes is always a working path
Zero-config magic becomes undebuggable	High	Every automatic decision logged and overridable; <code>orchestra</code> <code>doctor</code> makes it inspectable
Wrapped projects change or stall	Medium	Wrap behind our own interface; two queue backends supported so neither is load-bearing alone
Native dependencies without containers	Medium	Pinned machine images; lockfile hash verified at registration; containers remain supported
Auth deferred past the feature needing it	Critical	No remote-execution surface exists before Phase 5; auth ships in the same phase
Isolation density (2,336 KB vs 2 KB)	Medium	Accepted and designed around: subinterpreters are domains, not actors
Fork plus threads deadlock	Medium	Fork before any thread starts; free-threaded builds change this calculus
Debugger scope creep	Medium	Traceback and introspection first; live stepping is a later, separate decision
Beachhead saving too small to sell	Medium	Case B is unmeasured and uses a weaker lever than the headline. Measure on Linux before any positioning depends on it; the thesis does not rest on it.
Scope drifts back toward scheduling	Medium	§2's wrap-or-build table is a commitment, not a survey

The single most valuable early experiment. Before Phase 1, take one existing service and run it on a free-threaded build with threads instead of processes, and confirm every hot-path library has a free-threaded wheel. If RSS does not fall substantially, the thesis is wrong and week 7 will only confirm it more expensively. This costs about a day, and after the subinterpreter findings it is the load-bearing assumption remaining.